common thread here is producing a change in your project without a web interface. A significant percentage of Django commands are about bootstrapping, deploying, testing, and debugging your software.

The other common use for commands is to execute offline operations, such as cleaning up old files. As Django doesn't really run on its own but is activated through a web server, running an operation periodically can be problematic. There are tools out there that act as separate services to do this, but a quick way is to write a management command and then use your operating system's periodic execution mechanisms. On a Unix-based system, this would be `cron`, but other operating systems have equivalent services.

Say you wanted to use musician data from RiffMates in a utility script. You could write some raw SQL and access the database directly, but if the database changed, you'd have to change your script. Instead, you could write a management command that provided a querying mechanism for the musician data, which you could then call in your utility script. Your management command works within the context of your project, so you can use the same `Musician` `Model` you use in your views to perform your queries. In this chapter, you'll be writing a management command that does just that.

As for cleaning up, recall how `ImageField` and `FileField` in a Model point to uploaded files. Removing an object with these fields does not remove the corresponding files. Occasionally, you'll want to check for any orphaned files to save on disk space. A short management command is all that is needed to compare the contents of the uploads directory with the database. Writing such a management command is part of the exercises in this chapter. You'll have everything you need to try it yourself by the time you get there.

9.1.1 Built-in management commands

You can see all the available management commands for your project by running the `manage.py` script without any arguments. Its output is organized by Django app. When you write your own later, a new section will appear. Django has several of its own apps built in, so in addition to the `django` section, you'll see commands grouped under `auth`, `contenttypes`, `session`, and `staticfiles`.

> **NOTE** Django's terminology is a bit inconsistent; the `manage.py` script refers to these as "subcommands," while the documentation simply calls them "commands." Don't be confused; they're the same thing. Table 9.1 contains a summary of the built-in management commands and their purpose.

Table 9.1 Common management commands that are built into Django

Command	App	Description
`changepassword`	auth	Change a user's password. Doesn't require authentication.
`check`	django	Runs a check on the project. This is the same check the development server runs on startup.